10

DEEP Q-NETWORK

EL PROBLEMA DE Q-LEARNING

- Q-Learning se basa en el uso de una tabla de consulta (Q Table) donde para cada estado posible, la tabla le dice al agente qué acción devuelve la mejor recompensa futura esperada.
- Si estando en el estado s3 se toma la accion a1, la recompensa es de 14.
- Si se elige la acción a3 se obtiene una recompensa de 97.
- Q-Learning busca siempre la accion de le otorga al agente la máxima recompensa.

$$
\begin{array}{cc}
 & \begin{array}{ccccc} a0 & a1 & a2 & a3 & a4 \end{array} \\
Q = & \begin{bmatrix} 12 & 85 & 0 & 33 & 59 \\ 0 & 100 & 68 & 0 & 90 \\ 26 & 0 & 73 & 60 & 5 \\ 88 & 14 & 0 & 97 & 0 \\ 3 & 76 & 64 & 0 & 49 \end{bmatrix} \begin{array}{c} s0 \\ s1 \\ s2 \\ s3 \\ s4 \end{array}
\end{array}
$$

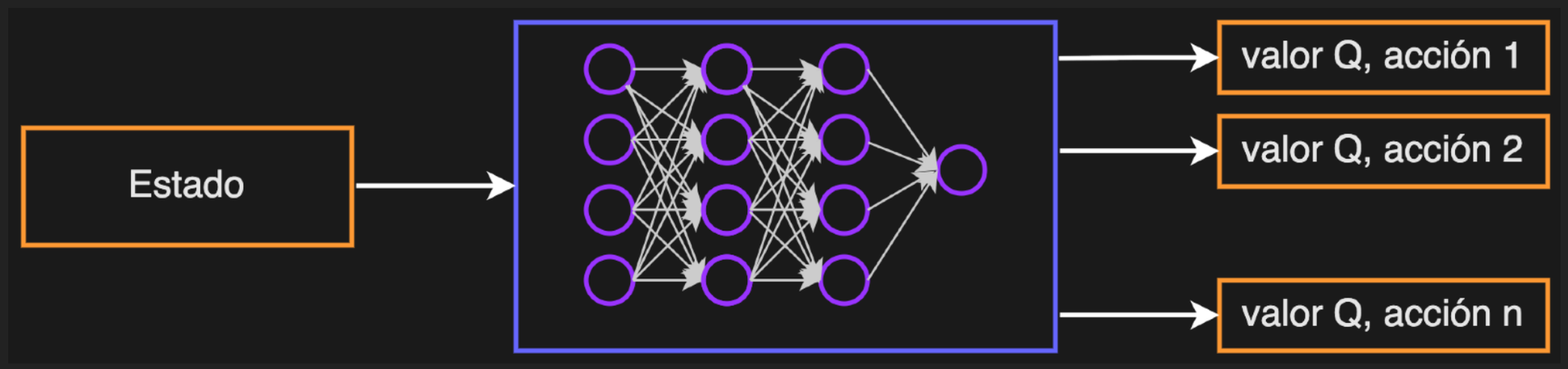
EL PROBLEMA DE Q-LEARNING

- ¿Qué sucede cuándo se tienen millones o miles de millones de estados?
- En ese caso, la tabla se vuelve inmanejable dado que se no puede almacenar o buscar en ella eficazmente debido a limitaciones de capacidad de memoria.
- Para solucionar esta limitación surgen las redes neuronales que son aproximadoras de funciones complejas (como la función de valor Q) y reemplazan a esta tabla gigante.
- Las redes aprenden la relación entre el estado, la acción y aproximan la recompensa esperada. Es decir que generalizan pudiendo asi abordar problemas complejos y de alta dimensionalidad en el espacios de estados.

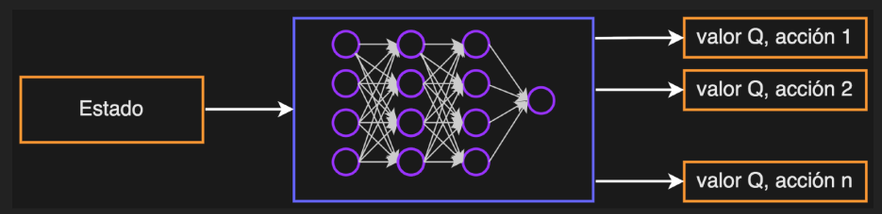
$$
\begin{array}{cc}
 & \begin{array}{ccccc} a0 & a1 & a2 & a3 & a4 \end{array} \\
Q = & \begin{bmatrix} 12 & 85 & 0 & 33 & 59 \\ 0 & 100 & 68 & 0 & 90 \\ 26 & 0 & 73 & 60 & 5 \\ 88 & 14 & 0 & 97 & 0 \\ 3 & 76 & 64 & 0 & 49 \end{bmatrix} & \begin{array}{c} s0 \\ s1 \\ s2 \\ s3 \\ s4 \end{array}
\end{array}
$$

DQN INICIAL

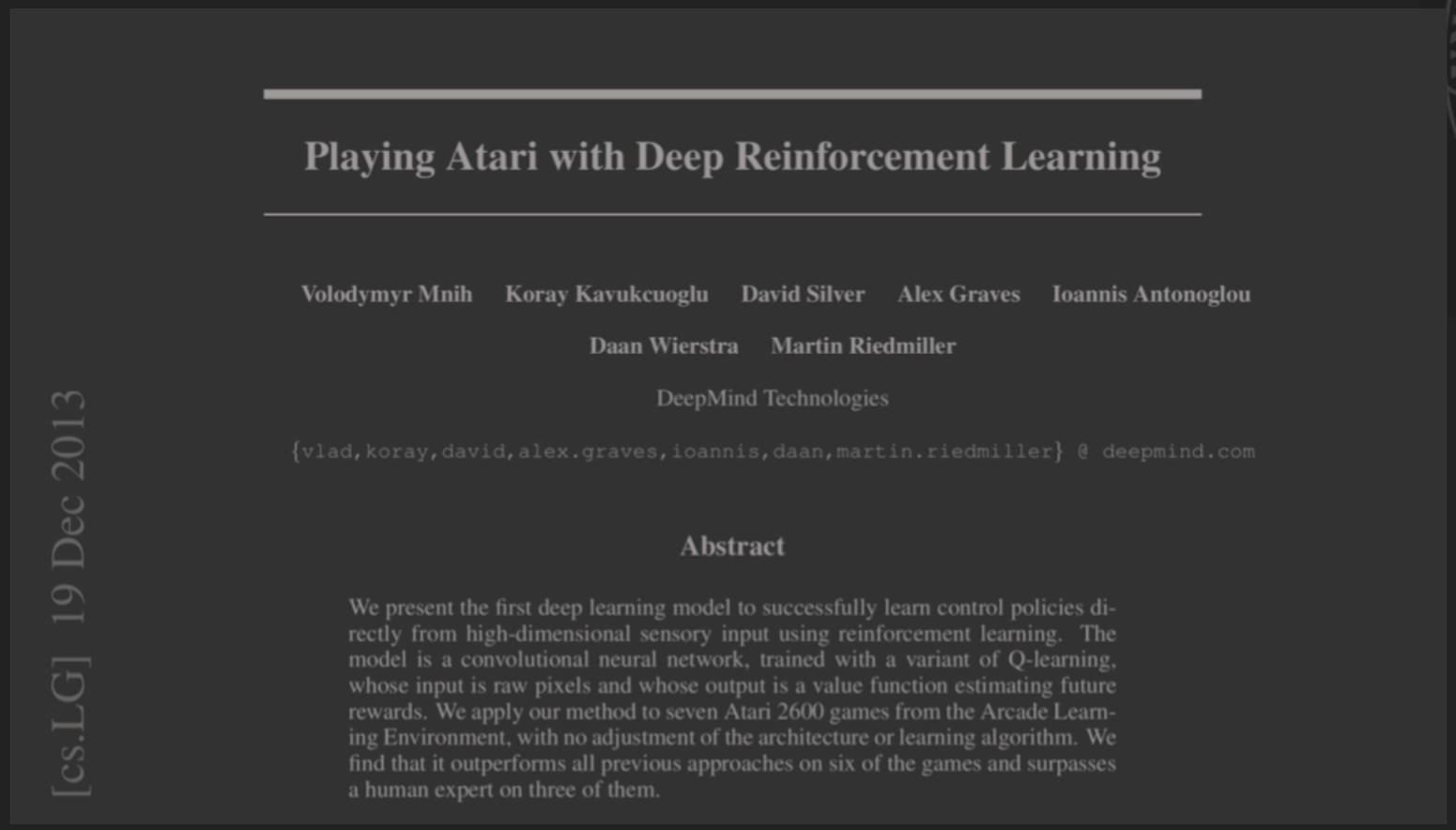
- Esta solución se conoce como DQN o redes Q profundas. Es decir, Q-learning más aprendizaje profundo.
- DQN intenta aprender el valor Q (recompensa futura esperada) pero usando una red neuronal profunda como aproximador de funciones en lugar de una tabla.



DQN INICIAL



- El articulo "Playing Atari with Deep Reinforcement Learning" propuso en 2013 este enfoque (Mnih et al., 2013).



Playing Atari with Deep Reinforcement Learning

Volodymyr Mnih Koray Kavukcuoglu David Silver Alex Graves Ioannis Antonoglou
Daan Wierstra Martin Riedmiller

DeepMind Technologies

{vlad,koray,david,alex.graves,ioannis,daan,martin.riedmiller} @ deepmind.com

[cs.LG] 19 Dec 2013

Abstract

We present the first deep learning model to successfully learn control policies directly from high-dimensional sensory input using reinforcement learning. The model is a convolutional neural network, trained with a variant of Q-learning, whose input is raw pixels and whose output is a value function estimating future rewards. We apply our method to seven Atari 2600 games from the Arcade Learning Environment, with no adjustment of the architecture or learning algorithm. We find that it outperforms all previous approaches on six of the games and surpasses a human expert on three of them.

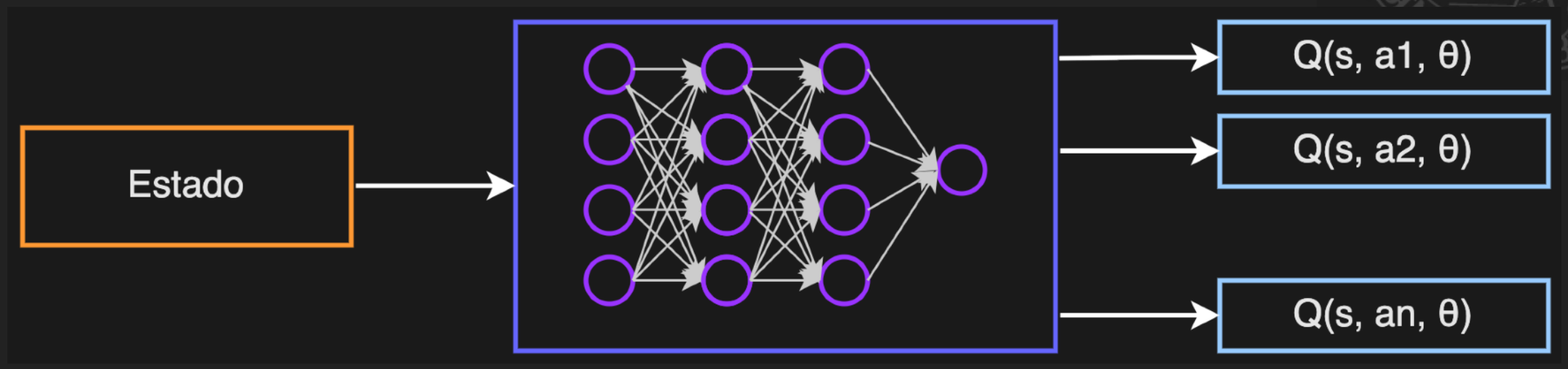
DQN INICIAL

- El artículo planteaba usar frames de algunos videojuegos de Atari 2600 como estados que ingresan a una red neuronal (3 capas convolucionales) y esta determina (estima) la función de valor Q, para que el agente tome la mejor acción.

DQN INICIAL

- Técnicamente la red va a predecir la función de valor Q que ademas de serena función del estado s y de la acción a también dependerá de los pesos θ que la red actualice.

- Q(s, a, θ) <– función de valor predicha por la red.

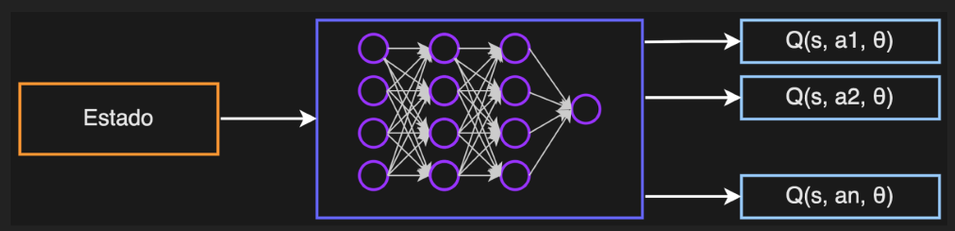


DQN INICIAL

- En DQN ya no se actualiza directamente $Q(s, a)$ en una tabla.
- En lugar de actualizar una tabla, se entrena una red neuronal para aproximar $Q(s, a, \theta)$.
- Para ello se usa una función de pérdida (loss) que se minimiza.

$$L = (y - Q(s, a; \theta))^2$$

- Donde:
  - ✓ $y = r + \gamma * \max_{a'} Q(s', a'; \theta)$ es el valor objetivo.
  - ✓ $Q(s, a; \theta)$ es el valor predicho por la red.
  - ✓ θ son los pesos de la red.
- Y luego se entrena la red con descenso de gradiente para minimizar la diferencia entre el valor actual y el objetivo y.



ARQUITECTURA DE DQN

- Los primeros modelos respondían a una estructura como la siguiente:

Inicializar red neuronal Q con pesos aleatorios θ

Para cada episodio:

Inicializar estado s

Mientras no sea estado terminal:

Con probabilidad ε elegir una acción aleatoria a

Si no, elegir a = argmax_a Q(s, a; θ)

Ejecutar acción a, observar recompensa r y nuevo estado s'

Calcular valor objetivo:

y = r + γ * max_a' Q(s', a'; θ) (si s' no es terminal)

y = r (si s' es terminal)

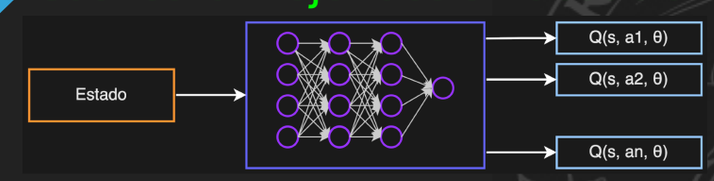
Calcular función de pérdida:

$L = (y - Q(s, a; \theta))^2$

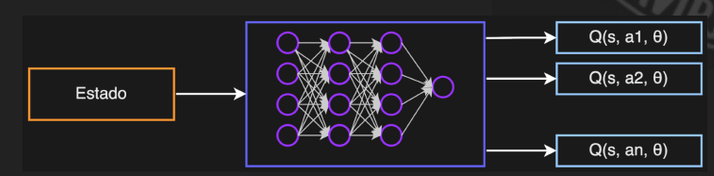
Actualizar pesos θ para minimizar L usando retropropagación

s ← s'

Cual es la mejor acción desde s?

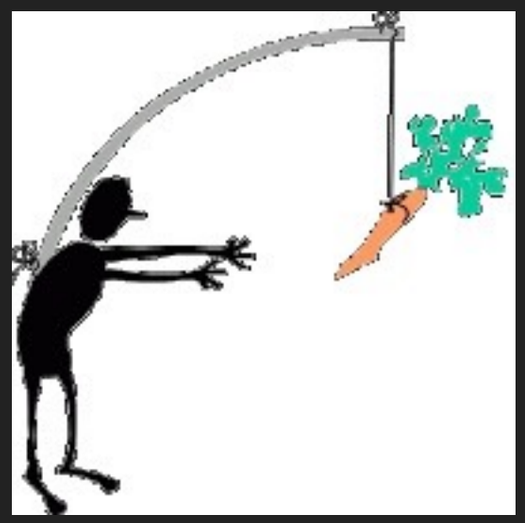


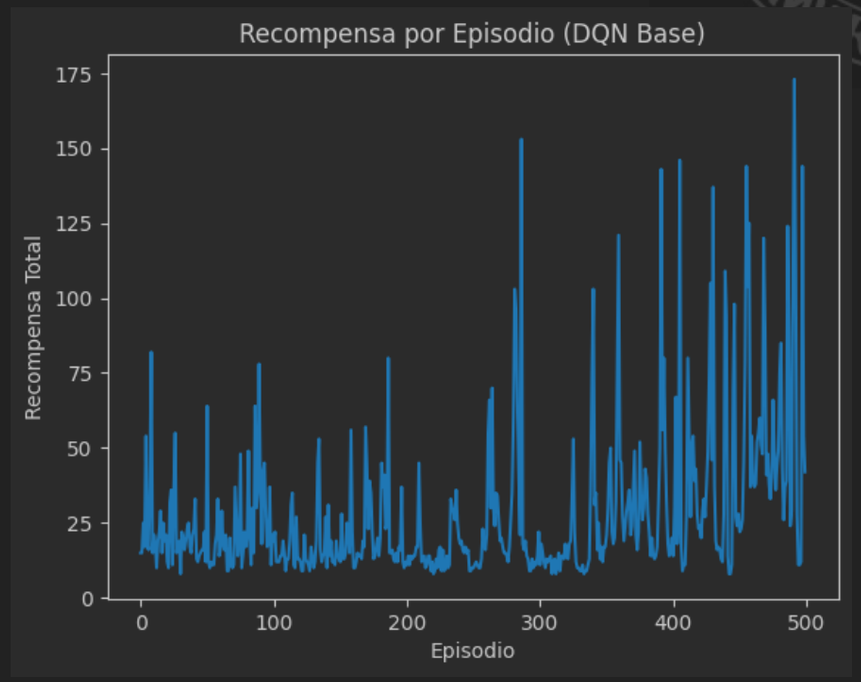
Cual es la mejor acción desde s'?



PROBLEMA 1 CON DQN INICIAL

- El valor objetivo y cambia constantemente en cada iteración.
- En aprendizaje supervisado normalmente el objetivo (lo que se intenta predecir), es fijo, pero en RL, los valores Q objetivo cambian a medida que el agente aprende.
- Analogía: Es como perseguir una meta que se mueve cada vez que se da un paso, en este caso:
  - ✓ El valor objetivo y depende de θ, que cambia en cada paso.
  - ✓ La red está intentando ajustarse a un objetivo que ella misma está moviendo.
  - ✓ Esto causa inestabilidad y oscilaciones durante el entrenamiento.



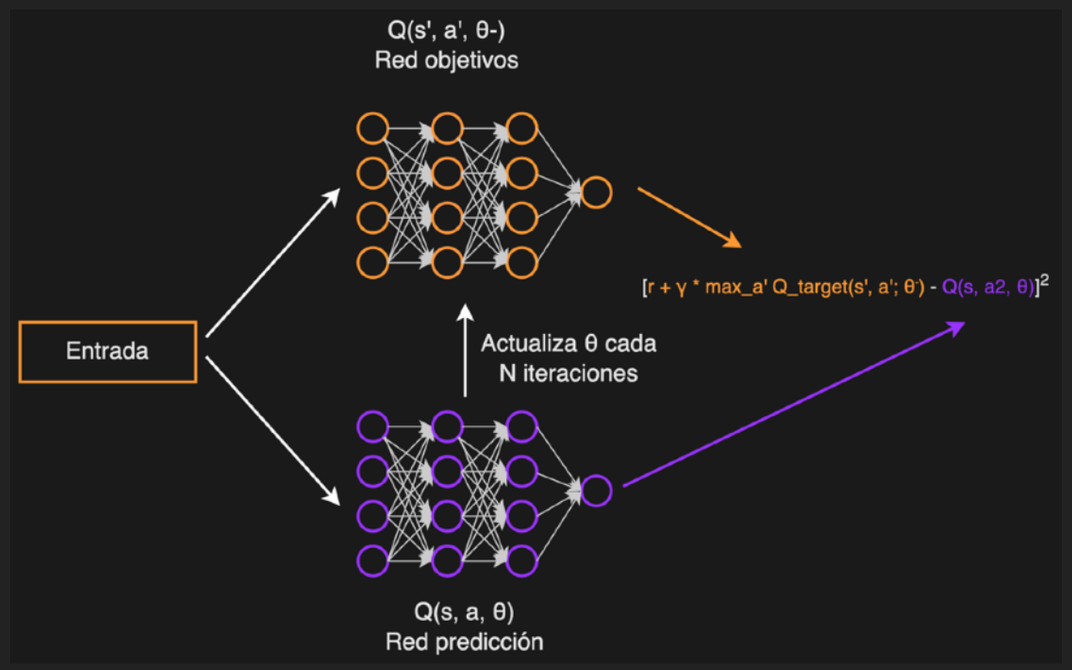


PROBLEMA 1 CON DQN INICIAL

- Solución: usar una segunda red, llamada red objetivo (Q_target) (Mnih et al., 2015)
- La red objetivo con pesos θ^- mantiene congelados los pesos por varios pasos de modo que el valor objetivo y:

$$y = r + \gamma * \max_a' \; Q_target(s', a'; \theta^-)$$

permanece más estable durante varias actualizaciones, permitiendo que la red principal tenga una meta fija temporalmente.

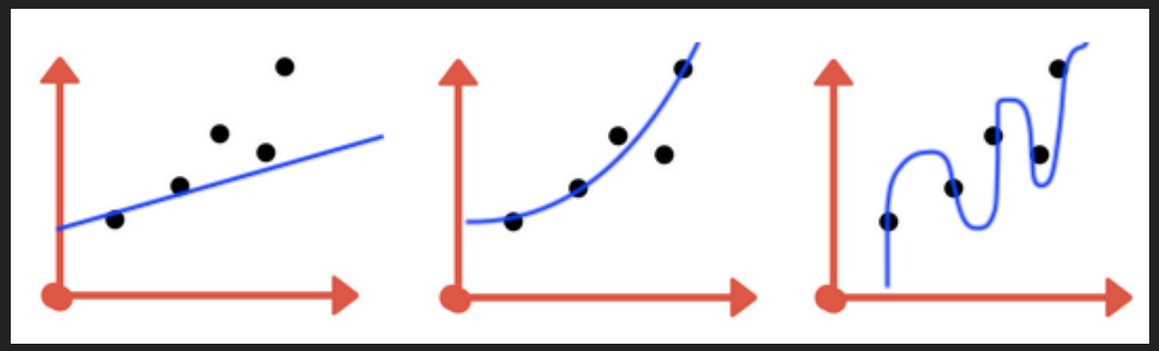


PROBLEMA 2 CON DQN INICIAL

- En RL, los datos son secuenciales es decir que están altamente correlacionados, no son muestras independientes.

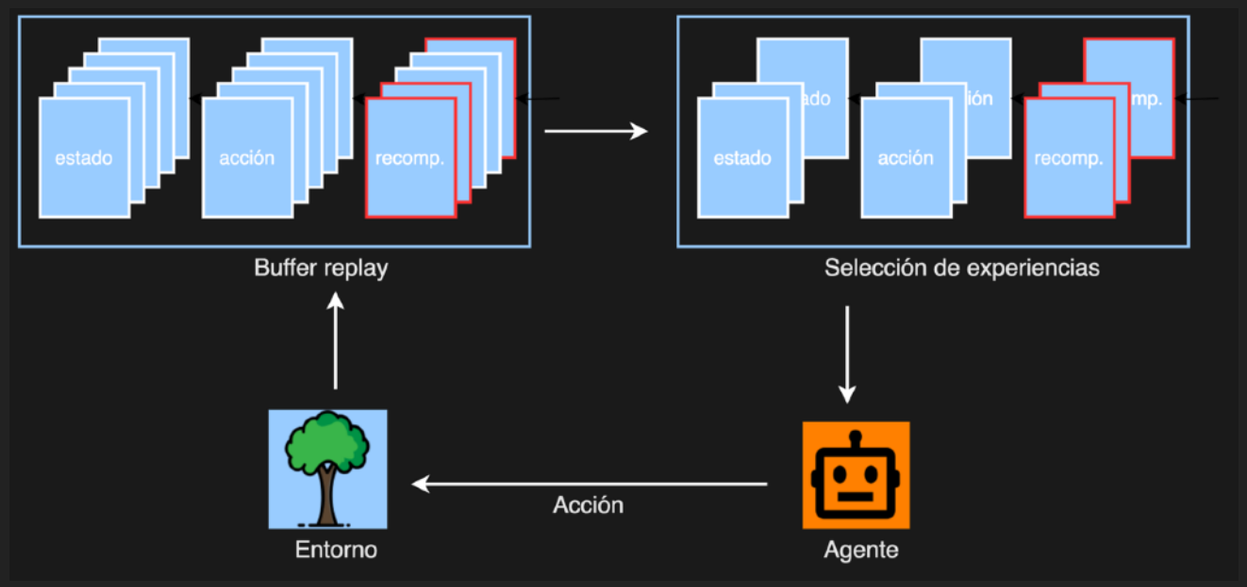
$$(s_0, a_0, r_0, s_1), (s_1, a_1, r_1, s_2), (s_2, a_2, r_2, s_3), \ldots$$

- Alimentar al mecanismo de descenso por gradiente estocástico (SGD) con muestras correlacionadas produce que la red:
  - ✓ sobreajusta a trayectorias locales y no generaliza (overfitting).
  - ✓ aprende patrones erróneos del orden de los datos en lugar de la política óptima, haciendo que el entrenamiento sea ruidoso o inestable.
  - ✓ puede divergir o quedarse atrapada en un subóptimo (oscila).
- Normalmente en aprendizaje supervisado las redes son entrenadas con el supuesto de que los datos son independientes e idénticamente distribuidos (i.i.d.); en DQN debiera suceder lo mismo.



PROBLEMA 2 CON DQN INICIAL

- Solución: Usar un banco de memoria donde se almacenan las transiciones o experiencias pasadas (tuplas estado, acción, recompensa, siguiente estado: S, A, R, S').
- Esta memoria se conoce como buffer replay y tiene generalmente un tamaño fijo.
- En este buffer Las experiencias más antiguas se eliminan a medida que entran nuevas (FIFO).
- En lugar de entrenar la DQN solo con la última transición, se muestrea aleatoriamente un pequeño lote (batch) de transiciones del búfer.



PSEUDOCÓDIGO DE DQN DEFINITIVO

Inicializar red neuronal Q (online) con pesos θ

Inicializar red Q_target con pesos $\theta^- \leftarrow \theta$

Inicializar replay buffer D vacío

Para cada episodio:

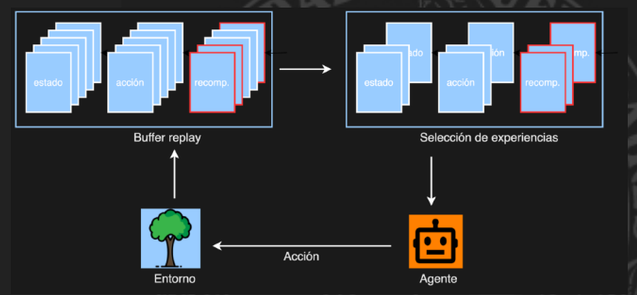
Inicializar estado s

Mientras no sea estado terminal:

Con probabilidad ε elegir una acción aleatoria a , si no, elegir a = argmax_a Q(s, a; θ)

Ejecutar acción a, observar recompensa r y nuevo estado s'

Almacenar transición (s, a, r, s') en el replay buffer D



Si hay suficientes muestras en D:

Extraer un minibatch aleatorio de transiciones desde D

Para cada transición (s, a, r, s') en el minibatch:

$y = r + \gamma * \max_a' Q_target(s', a'; \theta^-)$ (si s' no es terminal) , si no, $y = r$ (si s' es terminal):

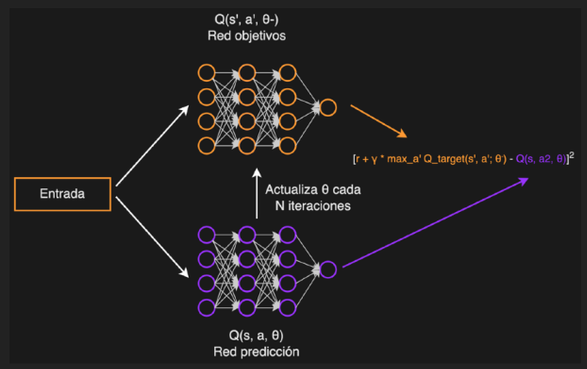
$L = (y - Q(s, a; \theta))^2$

Actualizar pesos θ para minimizar L usando retropropagación

$s \leftarrow s'$

Cada C pasos:

Actualizar red Q_target: $\theta^- \leftarrow \theta$



VARIANTES RELEVANTES DE DAN

- Double Q-network (Hasselt et al., 2015)
- Dueling Q-network (Wang et al., 2016)
- DQN categórica (Bellemare et al., 2017)
- Rainbow DQN (Hessel et al., 2017)

REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602.
- Mnih, V.; Kavukcuoglu, K.; Silver, D; et al. (2015). "Human-level control through deep reinforcement learning". Nature Journal.
- Wang, Z.; Schaul, T.; Hessel, M.; et al. (2016). Dueling Network Architectures for Deep Reinforcement Learning. Google DeepMind.
- Schaul, T.; Quan, J.; Antonoglou, I.; et al. (2016). Prioritized Experience Replay. Google DeepMind
- Hessel, H.; Modayil, J.; van Hasselt, H.; et al. (2017). Rainbow: Combining Improvements in Deep Reinforcement Learning. Google DeepMind.
- Hasselt, H. van; Guez, A.; Silver, D. (2015). Deep Reinforcement Learning with Double Q-learning. Google DeepMind.